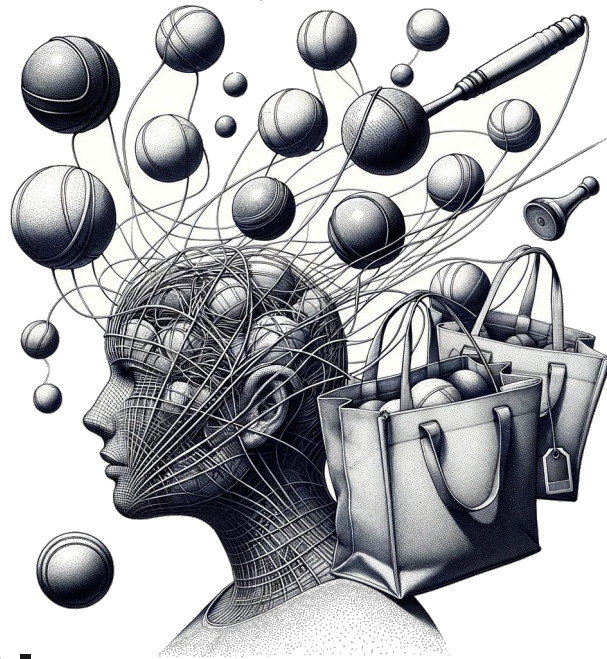




HypergraphAI

A Semantic Knowledge Hypergraph Platform for Enterprise Architecture

Comprehensive overview of HypergraphAI core concepts, system and container architecture, storage abstraction, SHQL query engine, security, MCP agent integration, HgNexus chat, and enterprise reference architectures.



Enterprise AI Requires Structured & Temporal Knowledge Stores

TRADITIONAL STORES FALL SHORT

Enterprise AI agents reading, reasoning, and writing across systems hit core limitations with legacy database paradigms:

- **Vector Indexes:** Retrieve by mathematical similarity rather than explicit meaning or facts.
- **Relational Tables:** Hide deep entity relationships across complex, costly joins.
- **Binary Edge Limits:** Multi-party real events force artificial intermediate nodes.
- **Opaque Reasoning:** Lacks explicit provenance and clear boundaries between stored and inferred facts.

HYPERGRAPHAI ARCHITECTURE

Purpose-built knowledge structure providing rich semantics, temporal tracking, and full explainability:

- **Hypernodes & Hyperedges:** First-class n-ary relationships with attributes, validity, tags, and member edge nesting (hub & symmetric).
- **Axiom Hyperedges:** Relation semantics stored directly as data (`owl:inverse-of`, `owl:symmetric`, `owl:transitive`).
- **Temporal Validity:** Native `valid_from` and `valid_to` properties on every node and edge.
- **Auditability:** Per-record provenance guarantees complete source lineage for agent reasoning.



Semantic Hypergraphs Uniquely Combine Native N-ary Modeling, Axiom Rules, and Agent Tool Interfaces

TRADITIONAL STORES (RELATIONAL / PROPERTY / VECTOR)

N-ary Facts & Attributes: Relational stores require joining multiple tables; property graphs rely on edge properties or extra columns.

Rules & Logic: Logic depends on database views, stored procedures, or complex external application plugins.

Temporal Queries: Requires manual implementation of complex bitemporal data modeling patterns.

Retrieval & Scale: Vector DBs excel at fuzzy retrieval. Relational and vector stores scale seamlessly across distributed systems.

HYPERGRAPHAI PARADIGM

N-ary Facts & Attributes: Handles multi-entity relationships, validity, and attributes natively using hyperedges.

Rules & Logic: Native support for rules as data via declarative axiom hyperedges.

Temporal Queries: Built-in "as of" temporal queries using the native **at:** parameter.

Retrieval, Scale & Agents: Lacks native fuzzy retrieval (best paired with a vector store). Operates on a single MongoDB backend; includes out-of-the-box MCP server with 30 tools for agents.

SELECTION CRITERIA: Choose HypergraphAI when value lies in rich relationships with meaning, time, and provenance shared seamlessly by humans and autonomous agents.



Stateless API Tier & Container Architecture

HypergraphAI System Context (C4 Level 1) & Container Overview (C4 Level 2)

C4 Level 1: System Context

External Users & Interfaces

Analysts and AI agents interact with the platform engine via Web UI, hgsh, MCP, or REST interfaces.

Platform Engine Integrations

The core engine acts as the central coordinator, connecting seamlessly to mesh servers and downstream dependencies.

External Dependencies

Requires MongoDB 7+ for primary state, with optional S3-compatible object storage for media bytes and AI vendor APIs for HgNexus integration.

INTERFACES:

[Web UI] [hgsh] [MCP] [REST API]

C4 Level 2: Container Architecture

FastAPI / Uvicorn Single Process

Encapsulates a static single-page Web UI, REST routers for entities & modules, and an MCP streamable-HTTP app (FastMCP) with auth middleware.

Core Processing & Operations

The internal core engine directly manages CRUD operations, inference tasks, and full audit logging pipelines.

Storage Abstraction & Async Driver

A dedicated storage abstraction layer interfaces directly with MongoDB using the Motor async driver.

Stateless Horizontal Scaling

State is maintained entirely within MongoDB, enabling the entire API tier to remain fully stateless and scale horizontally.



Decoupled Modules & Strict Storage Abstraction Layer

- **Decoupled Module Architecture:** Includes `hgai_core`, `hgai_module_storage`, `hgai_module_storage_mongodb`, `hgai_module_shql`, `hgai_module_mesh`, `hgai_module_mcp`, and `hgai_module_agentchat`. Modules mount at startup; a module that fails to import logs a warning and is skipped, allowing the core to keep running.
- **Storage Abstraction Contract:** Ensures all persistence routes strictly through `get_storage()`, preventing outside modules from importing MongoDB directly.
- **StorageBackend ABC & Interfaces:** Defines 10 per-entity store interfaces, such as `HypergraphStore`, `HypernodeStore`, and `HyperedgeStore`, along with typed filters and patch dataclasses.
- **Persistence & Media Storage Implementations:** MongoDB is currently the only implemented backend, utilizing the Motor async driver. Media bytes can be stored in GridFS by default or an AWS S3-compatible backend.
- **Extensibility Guarantee:** This strict contract allows connectors and ontology packs to be delivered as independent modules without requiring core modification.



Physical Data Model Enforces Edge Uniqueness via Deterministic SHA-256 Hyperkeys

LOGICAL DATA MODEL

Core Logical Entities

Consists of Mesh, Space (tenant), Hypergraph, Hypernode, Hyperedge, Account, Note, ParameterizedQuery, Media, and Agent Chat Session entities.

Ownership & Hierarchy

- A **Space (tenant)** owns a **Hypergraph**.
- Each **Hypergraph** contains underlying **Hypernodes** and **Hyperedges**.

ENTITY HIERARCHY FLOW

Space (Tenant) → Hypergraph → { Hypernodes, Hyperedges }

PHYSICAL MODEL & INDEX STRATEGIES

MongoDB Collections & Startup

Physical collections include hypergraphs, hypernodes, hyperedges, accounts, spaces, and meshes. Indexes are created idempotently at startup.

Node & Edge Indexing Strategies

- **Node Indexes:** Unique (id, hypergraph_id), status, type, tags, and sparse point-in-time (valid_from, valid_to).
- **Edge Strategy:** Unique (hyperkey, graph) index enforcing de-duplication at the database level.

SHA-256 HYPERKEY DE-DUPLICATION CONSTRAINT

Combines **relation + sorted members + graph + validity** to prevent duplicate hyperedges.

Identical relations and member sets are automatically merged into one edge with a list property.



Temporal Validity & Provenance Tracking

TEMPORAL SEMANTICS:

valid_from

valid_to

RECORD LIFECYCLE:

ACTIVE

DRAFT

ARCHIVED

at: PARAMETER

Temporal Querying & Staging Lifecycle

Temporal semantics are handled via `valid_from` and `valid_to` properties on every node and edge, enabling point-in-time (PIT) query resolution using the `'at:'` parameter in SHQL.

Data lifecycle management tracks status as active, draft, or archived, with draft serving as a staging state for agent-proposed facts.

Engine Graph Counts & Agent Stamping

The engine automatically maintains `node_count` and `edge_count` on the graph during write operations.

Writes through MCP tools stamp the audit trail with a generic `'mcp-agent'` actor, requiring the real agent identity to be explicitly recorded in provenance.

Provenance & Change History Tracking

Provenance and complete change history are maintained through a structured mutations array on each record, capturing full source lineage and audit trails.

Mutations Array Schema:

- **Timestamp & Actor:** Exact execution time and agent ID.
- **Mutation Details:** Mutation type and field-level delta.

Source Lineage Convention (`attributes.provenance`):

- **File Lineage:** Source file name and exact CSV line offset.
- **Execution Context:** Ingestion method name and unique run ID.



SHQL Query Execution & Pipeline Architecture



Evaluation & Pattern Constraints

- Evaluates **where**: patterns sequentially over binding sets.
- Supports Node/Edge, MongoDB filters, OPTIONAL & UNION.
- Shared variables trigger implicit joins; supports **at**: PIT.

Projection & Engine Optimizations

- Projections: select, distinct, count, group_by, order_by.
- Pagination: Limit default 500 | Max 2,000 docs/pattern.
- **Optimization**: Place selective predicates (type, tags, ids) first to avoid full scans from unindexed attributes.

```
query_execution_spec.shql

# SHQL Query Pipeline Structure
from: "mesh.local.analytics"
at: "2026-09-21T00:00:00Z"
where:
# Indexed predicates placed first
- node: "$user"
type: "User"
tags: ["active", "verified"]
- edge: "$user → $action → $event"
- optional:
filter: { "status": "COMPLETED" }
select: ["$user", "count($event)"]
group_by: ["$user"]
order_by: ["count($event) DESC"]
offset: 0
limit: 500 # Bound: 2000 max
```



Ontology Rules as Data & Fixed-Point Inference

Axiom Rules & Metadata Attribution

- **Axiom Hyperedges as Data:** Native support for `owl:inverse-of`, `owl:symmetric`, `owl:transitive`, `skos:broaderTransitive`, and `skos:narrowerTransitive`.
- **Metadata Attribution:** Every derived edge carries full provenance tracing, specifically `_inferred`, `_source_edge`, `_axiom`, and `_transitive_path`.

Execution Engine & Access Paths

- **Fixed-Point Expansion:** Performs bounded iterations with atomic fact de-duplication to guarantee cyclic axiom graphs terminate reliably.
- **Three Access Paths:** Query via SHQL `infer: true`, single edge expansion via REST/MCP (`/infer/expand`), and reachability checks (`/infer/transitive`).

Materialization Strategy: Project Inference

Because on-demand query evaluation (`infer: true`) evaluates from a bounded candidate set and can be slow on large-scale graphs, **Project Inference** serves as a dedicated materialization strategy.

It previews and writes derived facts back into the graph as ordinary hyperedges, making inference results fully indexable and exceptionally cheap to query at scale.



The Mesh Architecture Enables Federated Queries Across Domain Servers Without ETL

Mesh Registry & Graph Addressing

The federation architecture relies on a mesh registry containing server details such as `server_id`, `URL`, `API token`, and known graphs.

Graph addressing uses dot-notation, such as `mesh.server.space.graph`, to target remote data.

The system maintains background synchronization of each mesh's graph list based on a configurable interval defaulting to 300 seconds.

Execution, Fault Tolerance & Trust

Distributed execution is handled via asynchronous concurrent fan-out using a pooled `httpx.AsyncClient` with 100 connections and a 10-second timeout.

The failure model provides partial result fault tolerance, returning per-server errors rather than failing all-or-nothing.

However, cross-server query planning or pushdown is not supported beyond forwarding the query, and trust relies on a stored token per remote server which must be treated as a credential.



Security Architecture & Authorization Boundaries

Critical Authorization Gap Identified

POST /shq1/query and all **30 MCP tools** authenticate the caller but fail to check per-graph authorization. A readonly account can read every graph via SHQL and create nodes through MCP.

Authentication & Active RBAC

- **Token & Key Auth:** JWTs (8h default expiration) and dual API keys (primary/secondary) mapping to a synthetic full-admin account.
- **Account Roles:** Standardized RBAC supporting admin, user, agent, and readonly.
- **Encryption:** Vendor API keys for HgNexus are encrypted at rest using Fernet.
- **Enforced REST Boundaries:** Per-graph/space RBAC and membership gates successfully secure endpoints for nodes, edges, inference, and notes.

Architecture Hardening Roadmap

- **Network Policy Isolation:** Restrict and enforce network isolation for all /mcp/ routes.
- **Privileged Credential Handling:** Treat all MCP-capable credentials as highly privileged access paths.
- **Edge TLS Termination:** Terminate TLS in front of API services to centralize inspection and transport security.
- **Physical / Logical Isolation:** Deploy completely separate server instances for sensitive tenant data.



HypergraphAI 30 MCP Tool Surface & Risk Tier Architecture

SDK: Python FastMCP (ASGI @ /mcp/) **MODE:** stateless_http (Load Balancer Ready) **FRAMING:** JSON-RPC 2.0 over POST **AUTH:** Bearer Token Middleware

1. Read & Federation Tools

LOW RISK TIER

Tools: `query_execute`, `hypergraph_list`, `mesh_query`

Exposes non-mutating graph execution and distributed mesh discovery capabilities.

Returns errors as tool output (HTTP 200) so AI models can self-correct dynamically.

2. Write Data Tools

MEDIUM RISK TIER

Tools: `hypernode_create`, `media_upload`

Handles creation of graph hypernodes and binary media payloads without mutating existing state.

Protected by Bearer token authentication middleware prior to invoking tool execution.

3. Destructive Tools

HIGH RISK TIER

Tools: `hypernode_delete`

Executes permanent object removal and destructive graph state mutations.

Requires upstream gateway policy checks due to lack of per-user tool-level authorization.

4. Structure & Admin Tools

ADMIN TIER

Tools: `space_create`

Manages structural topology and multi-tenant space provisioning across system instances.

Gateway policy enforced in front of endpoint to strictly regulate execution by agent roles.



Agents Follow a Strict Discover-Query-Reason-Write Loop



01. Discover

Discover schemas; vector store finds initial candidates for hypergraph expansion.



02. Query

Validate query structures and execute targeted SHQL queries against graph.



03. Reason

Infer missing facts, synthesize context, and evaluate graph relationship paths.



04. Write & Verify

Write back findings as draft hyperedges with provenance; verify execution.

Integration & Topology

Supports IDE/desktop agents, HgNexus, LangGraph/Agno, batch, and GraphRAG hybrids.

Topology Guardrails:

- Enforces least privilege access
- Dedicated servers partitioned by data sensitivity tier

Safe Writes & Reviewability

All agent mutations are governed by strict draft tags and full audit trails.

status: draft
tags: [proposed-by-agent]

Auditability: Verified via mutations arrays and Point-In-Time (PIT) queries.

Future Platform Guardrails

Upcoming platform governance features for enterprise enforcement:

- **MCP Authorization:** Granular per-caller tool permissions
- **Identity Stamping:** Immutable per-agent provenance tracking
- **Usage Metering:** Real-time token and query quota controls



HgNexus Embedded Agent Chat Architecture



HgNexus Chat Engine Core

Features an **embedded browser panel** communicating via an **SSE streaming endpoint** with multi-vendor support for **Anthropic**, **OpenAI**, and **xAI** using decrypted server-side BYO keys.

Per-Turn Agent Toolkits

During each turn, the platform builds a custom agent equipped with three integrated toolkits:

- **Direct MCP Access**
In-house streamable-HTTP client. MCP tools inside a turn are not permission-scoped to the user.
- **Help Toolkit**
Account-aware in-process search.
- **Web Toolkit**
SSRF-guarded `web_fetch` integration.

Memory & Session Management

- **Agno over MongoDB**
Memory state is managed via Agno over MongoDB, replaying the last 10 runs for conversational context.
- **Session Persistence**
Features owner-isolated persisted sessions that remain pinned to one single model.
- **Account History**
The system retains a 50-prompt account history server-side.

Answer Export & Telemetry

- **Save as Note**
Answers can be exported directly as a "Save as Note" document.
- **Structured Output Format**
Outputs Markdown formatted content paired with detailed YAML front-matter.
- **Captured Metadata:**
 - Original Prompt
 - LLM Vendor Name
 - Execution Timing
 - Total Token Counts



Deployment Topologies: Departmental to Federated Enterprise Meshes

01 / PILOT TOPOLOGY

Departmental Knowledge Store

Lightweight single-container deployment paired with MongoDB via Docker Compose. Optimized for initial team pilots and rapid use-case validation.

02 / ENTERPRISE MESH

Federated Enterprise Mesh

Multi-domain architecture where each business unit manages isolated MongoDB instances and domain-specific RBAC, queried seamlessly via a central hub server.

03 / HYBRID RETRIEVAL

GraphRAG Hybrid Agent Platform

Integrates a vector store for similarity retrieval alongside a hypergraph database to preserve deep structure, temporal dynamics, and lineage provenance.

04 / SECURE ISOLATION

Regulated Air-Gapped Deployment

Strict server isolation tailored per classification level with zero outbound dependencies. Deployed via Docker Compose or Kubernetes with stateless nodes.

SIZING & OPS: ~3.6M Nodes / 1.8M Edges on Single MongoDB • 12-Factor HGAI_* Env Config • K8s / Compose Ready